

# MIT

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

## Academy of Engineering

**Subject  
(Code):**  
Essentials of Data  
Science  
(2304102L)

# EDS CodeTantra Programs

| Name                 | (PRN)          | Div- Batch |
|----------------------|----------------|------------|
| TANMAY Rahul MUTKULE | (202501090093) | ME2 – M21  |



# Practical 01

## 1.1. Practice Lab Assignments

### 1.1.1. Calculate Momentum

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum  $p$  is calculated using the formula:

$$p = m v$$

where:

$p$  is the mass of the object (in kilograms).

$m$  is the velocity of the object (in meters per second).

#### Input Format:

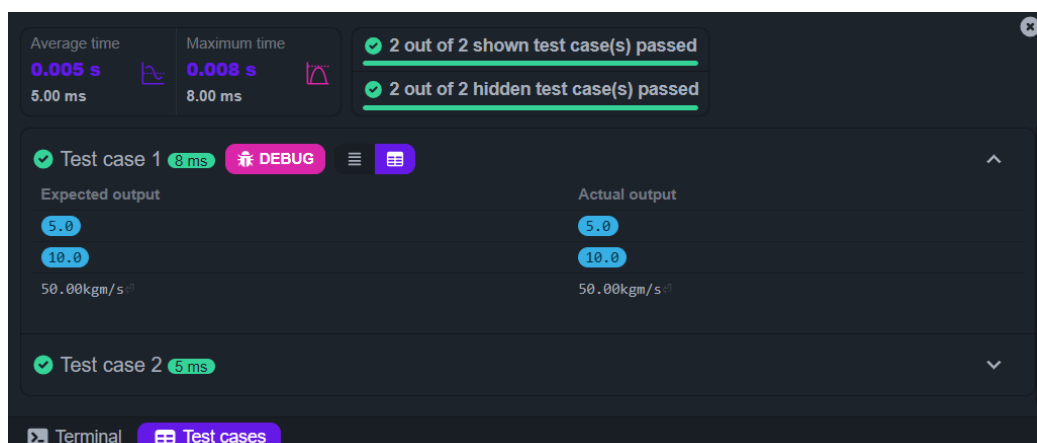
A single floating-point number representing the mass of the object in kilograms.

A single floating-point number representing the velocity of the object in meters per second.

#### Output Format:

The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

```
mass=float(input())
velocity=float(input())
momentum=mass*velocity
print(f"{momentum:.2f}kgm/s")
```



### 1.1.2 Conditional Calculation based on the Number of Digits

Write a Python program that accepts an integer as input. Depending on the number of digits in  $n$ .

**Constraints:**

$$1 \leq n \leq 999$$

**Input Format:**

The input consists of a single integer  $n$ .

**Output Format:**

If  $n$  is a single-digit number, print its square.

If  $n$  is a two-digit number, print its square root (rounded to two decimal places).

If  $n$  is a three-digit number, print its cube root (rounded to two decimal places).

Else print "Invalid".

```
import math

number=int(input())

if 0<= number <=9:

    print(number**2)

elif 10<= number <=99:

    print(f"{math.sqrt(number):.2f}")

elif 100<= number <=999:

    print(f"{math.cbrt(number):.2f}")

else:

    print("Invalid")
```



Average time

0.004 s

3.71 ms



Maximum time

0.005 s

5.00 ms



✓ 4 out of 4 shown test case(s) passed

✓ 3 out of 3 hidden test case(s) passed

✓ Test case 1

5 ms

🐞 DEBUG



Expected output

Actual output

9

9

81<sup>d</sup>

81<sup>d</sup>

✓ Test case 2

5 ms



✓ Test case 3

4 ms



📄 Terminal

📄 Test cases

### 1.1.3. Days Between Two Dates

Write a Python program to calculate the number of days between two dates.

#### Input Format:

The first line contains the first date in the format `YYYY-MM-DD`.

The second line contains the second date in the format `YYYY-MM-DD`.

#### Output Format:

An integer representing the number of days between the given dates.

```
from datetime import datetime
```

```
date1=input().strip()
```

```
date2=input().strip()
```

```
d1=datetime.strptime(date1,"%Y-%m-%d")
```

```
d2=datetime.strptime(date2,"%Y-%m-%d")
```

```
difference=d2-d1
```

```
print(difference.days)
```

The screenshot displays a code execution interface with a dark theme. At the top, performance metrics are shown: 'Average time' is 0.011 s (11.00 ms) and 'Maximum time' is 0.027 s (27.00 ms). To the right, two green checkmarks indicate '2 out of 2 shown test case(s) passed' and '2 out of 2 hidden test case(s) passed'. Below this, 'Test case 1' is expanded, showing a duration of 27 ms and a 'DEBUG' button. It compares 'Expected output' (2014-07-02, 2014-11-02, 123) with 'Actual output' (2014-07-02, 2014-11-02, 123). 'Test case 2' is collapsed, showing a duration of 5 ms. At the bottom, there are tabs for 'Terminal' and 'Test cases'.

### 1.1.4. Reverse a Number

Write a Python program to reverse the digits of the number and print the reversed number.

#### Input Format:

- The input is a single integer.

#### Output Format:

- The output prints a single integer, which is the reversed number.

```
n = int(input())
```

```
reversed_num=int(str(num)[::-1])
```

```
print(reversed_num)
```

Average time  
**0.004 s**  
4.00 ms

Maximum time  
**0.005 s**  
5.00 ms

✓ 2 out of 2 shown test case(s) passed

✓ 3 out of 3 hidden test case(s) passed

✓ Test case 1 5 ms DEBUG

| Expected output | Actual output |
|-----------------|---------------|
| 5367            | 5367          |
| 7635            | 7635          |

✓ Test case 2 5 ms

Terminal

Test cases

### 1.1.5. Multiplication Table

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

#### Input Format:

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

#### Output Format:

- Print the multiplication table for the given number.  
<number> x <multiplier> = <result>

#### Note:

- The character "x" is used in the output to represent multiplication.
- Refer to the visible test-cases for more clarity.

```
num = int(input())

for i in range(1, 11):
    print(f"{num} x {i} = {num * i}")
```

The screenshot displays a code execution interface with the following components:

- Performance Metrics:**
  - Average time: 0.004 s (4.25 ms)
  - Maximum time: 0.008 s (8.00 ms)
- Test Results:**
  - 2 out of 2 shown test case(s) passed
  - 2 out of 2 hidden test case(s) passed
- Test Case Details (Test case 1, 8 ms):**

| Expected output      | Actual output        |
|----------------------|----------------------|
| 8                    | 8                    |
| 8 · x · 1 · = · 8 ·  | 8 · x · 1 · = · 8 ·  |
| 8 · x · 2 · = · 16 · | 8 · x · 2 · = · 16 · |
| 8 · x · 3 · = · 24 · | 8 · x · 3 · = · 24 · |
| 8 · x · 4 · = · 32 · | 8 · x · 4 · = · 32 · |
| 8 · x · 5 · = · 40 · | 8 · x · 5 · = · 40 · |
- Navigation:** Buttons for 'Terminal' and 'Test cases' are visible at the bottom.

## 1.2 Lab Assignments

### 1.2.1. Pass or Fail

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

#### Input Format:

- The first input will be an integer  $n$ , the number of courses.
- The second input will be  $n$  integers representing the marks of the student in each of the  $n$  courses, separated by a space.

#### Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks  $< 40$  in any course), print:

- "Fail".

#### Note:

- Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing by the number of courses.

```
n = int(input())
```

```
marks= list(map(int,input().split()))
```

```
failed= False
```

```
for m in marks:
```

```
if m<40:
```

```
    failed=True
```

```
    break
```

```
if failed:
```

```
    print("Fail")
```

```
else:
```

```
    total= sum(marks)
```

```
    agg=total/n
```

```
    print("Aggregate Percentage: "f"{agg:.2f}")
```

```
if agg > 75:
```

```
    print("Grade: Distinction")
```

```
elif agg >=60:
```

```
    print("Grade: First Division")
```

```
elif agg>=50:
```

```
    print("Grade: Second Division")
```

```
elif agg >=40:
```

```
    print("Grade: Third Division")
```

Average time  
**0.007 s**  
6.70 ms

Maximum time  
**0.017 s**  
17.00 ms

✓ 5 out of 5 shown test case(s) passed

✓ 5 out of 5 hidden test case(s) passed

✓ Test case 1 14 ms **DEBUG**

Expected output

Actual output

4

4

55 65 78 89

55 65 78 89

Aggregate Percentage: 71.75

Aggregate Percentage: 71.75

Grade: First Division

Grade: First Division

✓ Test case 2 17 ms

Terminal

Test cases

### 1.2.2. Fibonacci Series using Recursive Function

Write a Python program that uses recursion to print the first n terms of the Fibonacci series.

#### Input Format:

A single integer n representing the number of terms of generate.

#### Output Format:

A single line containing the first n terms of the Fibonacci sequence, separated by spaces.

```
def fibonacci(n):  
    if n==1:  
        return 0  
    elif n==2:  
        return 1  
    else:  
        return fibonacci(n-1) + fibonacci(n-2)
```

```
n = int(input())  
for i in range(1, n + 1):  
    print(fibonacci(i), end=" ")
```

The screenshot displays a code execution environment with the following details:

- Performance Metrics:**
  - Average time: 0.011 s (10.50 ms)
  - Maximum time: 0.025 s (25.00 ms)
- Test Results:**
  - 2 out of 2 shown test case(s) passed
  - 2 out of 2 hidden test case(s) passed
- Test Case 1 Details:**
  - Status: Passed (7 ms)
  - Buttons: DEBUG, menu, list
  - Expected output: 5
  - Actual output: 5
  - Input sequence: 0 · 1 · 1 · 2 · 3 ·
- Test Case 2 Details:**
  - Status: Passed (4 ms)
- Navigation:** Terminal, Test cases

### 1.2.3. Pattern-1

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

#### Input Format:

- The input is an integer, representing the number of rows in the pattern.

#### Output Format

- The output should display the pattern of asterisks (\*), with each row containing an increasing number of asterisks.

**Note:** Refer to the displayed test cases for the sample pattern.

#### Test case 1:

4

\*.

\*.\*.

\*.\*.\*.

\*.\*.\*.\*.

```
n = int(input())
for i in range(1, n + 1):
    print('*' * i)
```

Average time  
**0.005 s**  
4.83 ms

Maximum time  
**0.011 s**  
11.00 ms

2 out of 2 shown test case(s) passed  
4 out of 4 hidden test case(s) passed

Test case 1 11 ms DEBUG

Expected output

Actual output

5

5

\*.

\*.

\*.\*.

\*.\*.

\*.\*.\*.

\*.\*.\*.

\*.\*.\*.\*.

\*.\*.\*.\*.

\*.\*.\*.\*.\*.

\*.\*.\*.\*.\*.

Terminal

Test cases

### 1.2.4. Pattern-2

Write a Python program to print a right-angled triangle pattern of numbers.

#### Input Format:

- The input is an integer, representing the number of rows in the pattern.

#### Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

#### Note:

- Refer to the displayed test cases for the sample pattern.

#### Test case 1:

5

1 ↵

1 2 ↵

1 2 3 ↵

1 2 3 4 ↵

1 2 3 4 5 ↵

```
n = int(input())
for i in range(1, 1+n):
    for j in range(1, 1+i):
        print(j, end=" ")
    print("")
```

Average time  
**0.004 s**  
4.25 ms



Maximum time  
**0.006 s**  
6.00 ms



✓ 2 out of 2 shown test case(s) passed

✓ 2 out of 2 hidden test case(s) passed

✓ Test case 1 2 ms

DEBUG





^

| Expected output                                                                                       | Actual output                                                                                         |
|-------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| 5                                                                                                     | 5                                                                                                     |
| 1 ·                  | 1 ·                  |
| 1 · 2 ·              | 1 · 2 ·              |
| 1 · 2 · 3 ·          | 1 · 2 · 3 ·          |
| 1 · 2 · 3 · 4 ·      | 1 · 2 · 3 · 4 ·      |
| 1 · 2 · 3 · 4 · 5 ·  | 1 · 2 · 3 · 4 · 5 ·  |

 Terminal

 Test cases